

```
In [3]: # IMPORT REQUIRED LIBRARIES

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import re
import nltk
import warnings

warnings.filterwarnings("ignore")

# NLP Libraries
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Machine Learning Libraries
from sklearn.model_selection import (train_test_split, cross_val_score, RandomizedSearchCV)
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix, precision_score, recall_score, f1_score)

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.ensemble import StackingClassifier

from xgboost import XGBClassifier

# Save Model
import pickle

In [4]: # DOWNLOAD NLTK STOPWORDS
nltk.download('stopwords')

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\sahil\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

Out[4]: True

In [5]: df = pd.read_csv('spam.csv', encoding='latin-1')

In [6]: df.head()

Out[6]:
```

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

```
In [7]: # Keep only useful columns
df = df[['v1', 'v2']]

In [8]: # Rename the columns
df.columns = ['label', 'message']

In [9]: df.head()

Out[9]:
```

	label	message
0	ham	Go until jurong point, crazy. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

```
In [10]: # Basic Data Inspection

In [14]: print("Dataset Shape\n", df.shape)

Dataset Shape
(5572, 2)

In [15]: print("Dataset Info\n", df.info())

<class 'pandas.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
--  --
 0   label   5572 non-null      str
 1   message 5572 non-null      str
dtypes: str(2)
memory usage: 541.4 KB
Dataset Info
None

In [18]: print("missing values\n", df.isnull().sum())

missing values
label      0
message    0
dtype: int64

In [20]: print("Duplicate\n", df.duplicated().sum())

Duplicate
483

In [21]: # REMOVE DUPLICATES

df.drop_duplicates(inplace=True)

print("dataset Shape after Removing Duplicates:", df.shape)

dataset Shape after Removing Duplicates: (5169, 2)

In [22]: # ENCODE TARGET VARIABLE
# ham = 0
# spam = 1
df['label'] = df['label'].map({
    'ham':0,
    'spam':1
})
df.head()

Out[22]:
```

	label	message
0	0	Go until jurong point, crazy. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

```
In [23]: #Exploratory Data Analysis (EDA)

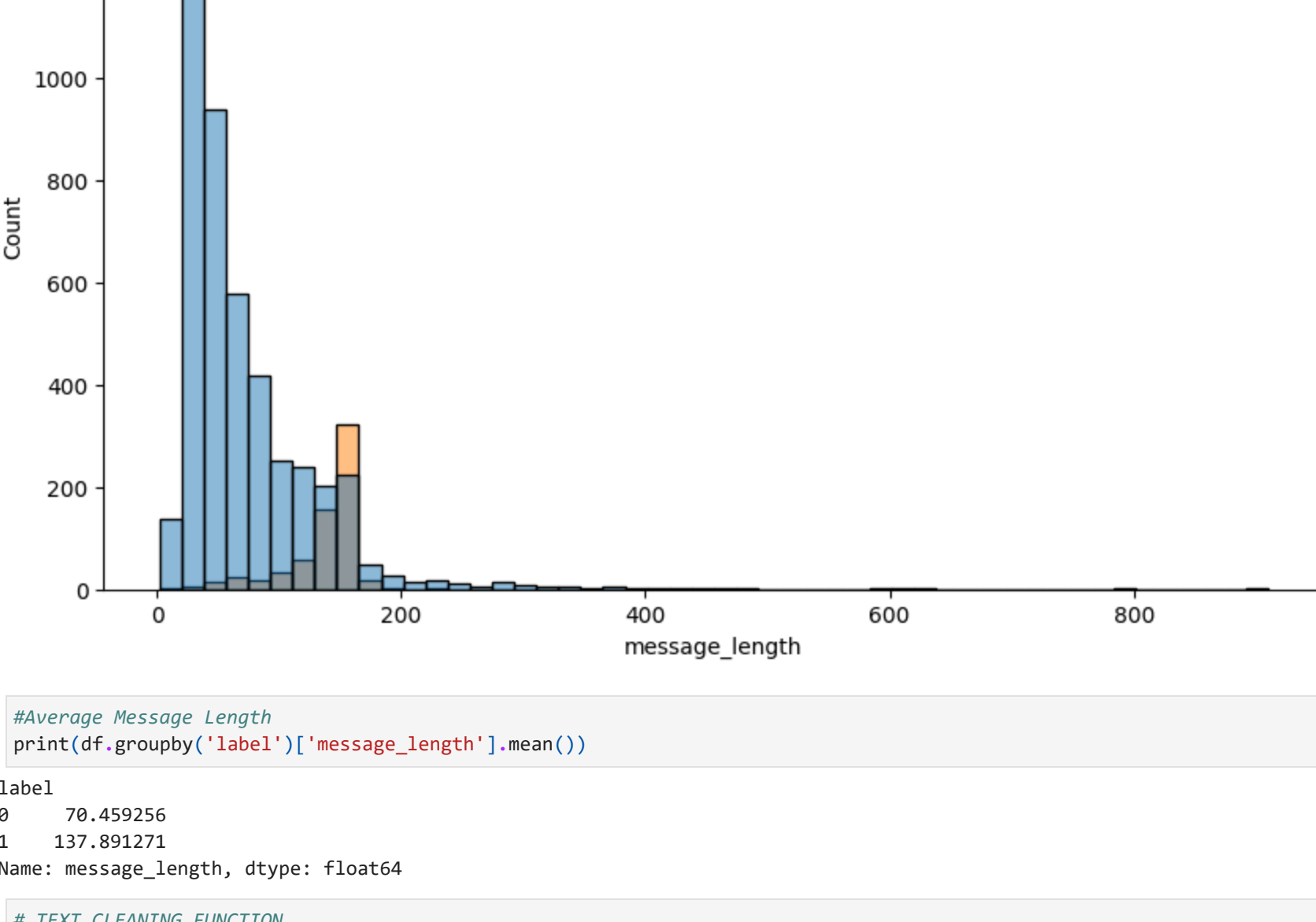
In [25]: # CLASS DISTRIBUTION

plt.figure(figsize=(8,4))
sns.countplot(x=df['label'])
plt.title("Spam vs Ham")
plt.xticks([0,1], ['Ham', 'Spam'])
plt.show()
```



```
In [27]: # MESSAGE LENGTH ANALYSIS

df['message_length'] = df['message'].apply(len)
plt.figure(figsize=(10,6))
sns.histplot(
    data=df,
    x='message_length',
    hue='label',
    bins=50
)
plt.title("Message Length")
plt.show()
```



```
In [28]: #Average Message Length
print(df.groupby('label')['message_length'].mean())

label
0    70.493256
1   137.891271
Name: message_length, dtype: float64

In [30]: # TEXT CLEANING FUNCTION
ps = PorterStemmer()

def clean_text(text):
    #Convert to lowercase
    text = text.lower()

    #Remove special characters
    text = re.sub(r'[^a-zA-Z]', ' ', text)

    #Tokenization
    words = text.split()

    # Remove stopwords + stemming
    words = [
        ps.stem(word)
        for word in words
        if word not in stopwords.words('english')
    ]

    return " ".join(words)

In [31]: # APPLY TEXT CLEANING

df['clean_message'] = df['message'].apply(clean_text)
df.head()

Out[31]:
```

	label	message	message_length	clean_message
0	0	Go until jurong point, crazy. Available only ...	111	go jurong point crazi avail bugli n great world...
1	0	Ok lar... Joking wif u oni...	29	ok lar joke wif u oni
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	free entri wkli comp win fa cup final txt st m...
3	0	U dun say so early hor... U c already then say...	49	u dun say earli hor u c already say
4	0	Nah I don't think he goes to usf, he lives aro...	61	nah think goe usf live aro...

```
In [32]: # FEATURE EXTRACTION USING TF-IDF

tfidf = TfidfVectorizer(
    max_features=10000,
    ngram_range=(1,2),
    min_df=2,
    max_df=0.95
)

In [33]: # DEFINE FEATURES & TARGET
X = df['clean_message']
y = df['label']

In [34]: # TRAIN TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

In [50]: #Build Advanced Ensemble Learning Model
# DEFINE BASE MODELS

# =====
# DEFINE BASE MODELS
# =====

base_models = [
    ('lr', LogisticRegression( class_weight='balanced', C=1, max_iter=1000)),
    ('svm', LinearSVC( class_weight='balanced' )),
    ('nb', MultinomialNB()),
    ('xgb', XGBClassifier( eval_metric='logloss', learning_rate=0.1, max_depth=5, n_estimators=200, subsample=0.8, colsample_bytree=0.8 )
)]

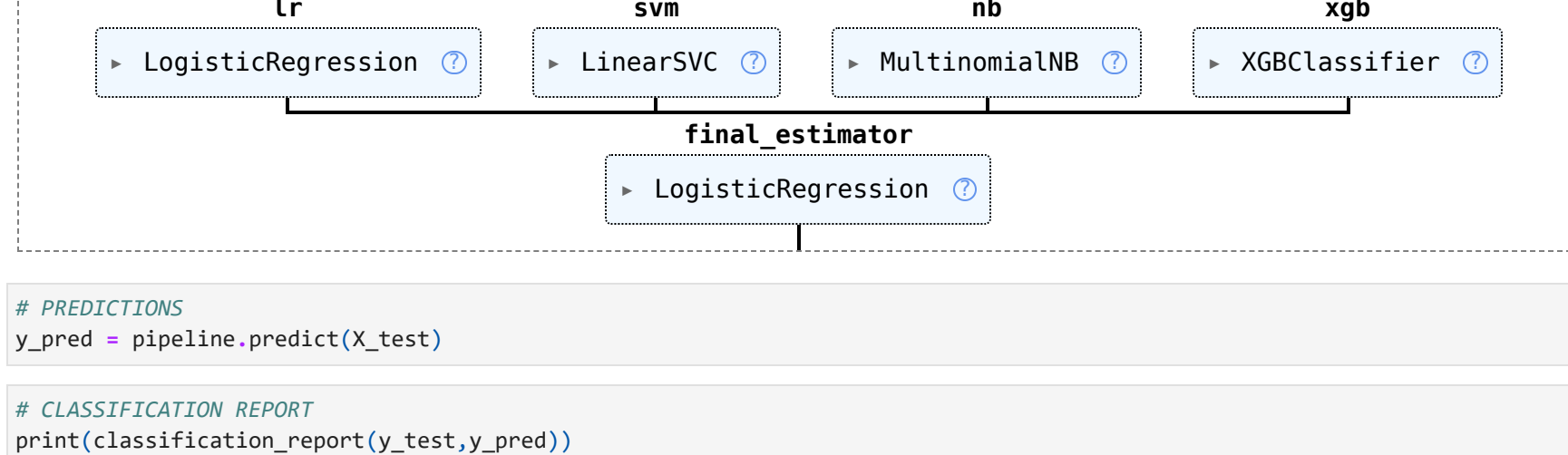
In [51]: # META MODEL
meta_model = LogisticRegression()

In [52]: # STACKING CLASSIFIER
stack_model = StackingClassifier(estimators=base_models, final_estimator=meta_model, cv=5, n_jobs=-1)

In [53]: # CREATE PIPELINE
pipeline = Pipeline([('tfidf', tfidf), ('model', stack_model)])

In [54]: pipeline.fit(X_train, y_train)

Out[54]:
```



```
In [57]: # PREDICTIONS
y_pred = pipeline.predict(X_test)

In [58]: # CLASSIFICATION REPORT
print(classification_report(y_test, y_pred))

precision    recall  f1-score   support

0           0.99         1.00         0.99         903
1           0.98         0.92         0.95         131

accuracy          0.99         0.96         0.99        1034
macro avg         0.99         0.96         0.97        1034
weighted avg         0.99         0.99         0.99        1034

In [59]: # PERFORMANCE METRICS

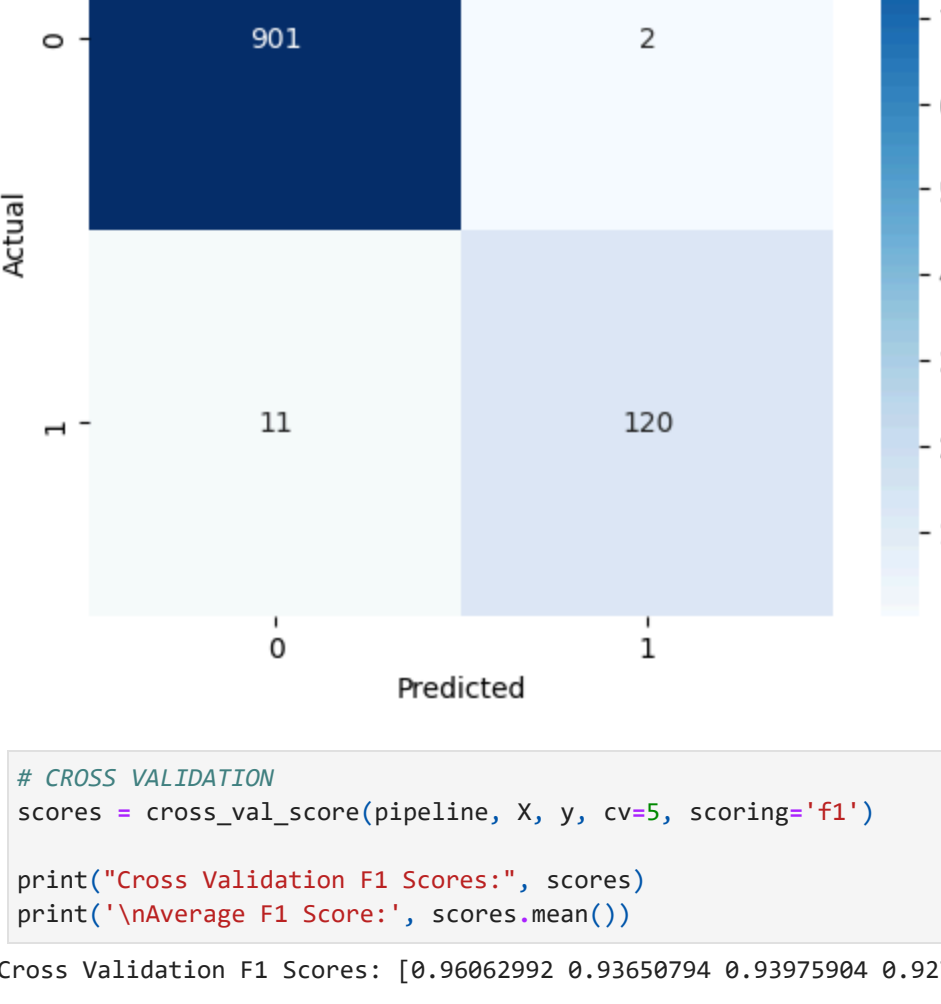
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

Accuracy: 0.987474661588784
Precision: 0.9836865573778492
Recall: 0.916898345351145
F1 Score: 0.9488166807985138

In [61]: # CONFUSION MATRIX
cm = confusion_matrix(
    y_test, y_pred
)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title("Confusion Matrix")
plt.show()
```



```
In [62]: # CROSS VALIDATION
scores = cross_val_score(pipeline, X, y, cv=5, scoring='f1')

print("Cross Validation F1 Scores:", scores)
print("Average F1 Score:", scores.mean())

Cross Validation F1 Scores: [0.96062992 0.93658794 0.93975984 0.92741935 0.92687884]

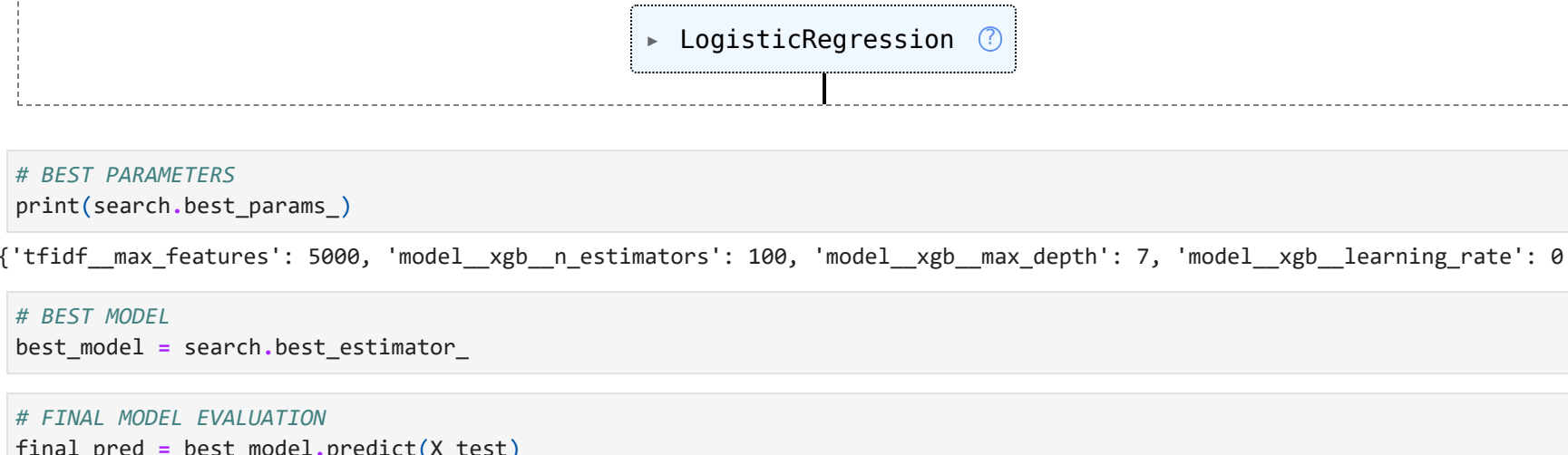
Average F1 Score: 0.938077257523146

In [63]: # HYPERPARAMETER TUNING
params = {
    'tfidf_max_features': [5000, 10000],
    'model_xgb_max_depth': [3,5,7],
    'model_xgb_learning_rate': [0.01, 0.1],
    'model_xgb_n_estimators': [100,200]
}

search = RandomizedSearchCV(
    pipeline, params, n_iter=10, cv=5, scoring='f1', n_jobs=-1, random_state=42
)

search.fit(X_train, y_train)

Out[63]:
```



```
In [64]: # BEST PARAMETERS
print(search.best_params_)

{'tfidf_max_features': 5000, 'model_xgb_n_estimators': 100, 'model_xgb_max_depth': 7, 'model_xgb_learning_rate': 0.01}

In [65]: # BEST MODEL
best_model = search.best_estimator_

In [66]: # FINAL MODEL EVALUATION
final_pred = best_model.predict(X_test)

print(classification_report(y_test, final_pred))

precision    recall  f1-score   support

0           0.99         1.00         0.99         903
1           0.98         0.92         0.95         131

accuracy          0.99         0.96         0.99        1034
macro avg         0.99         0.96         0.97        1034
weighted avg         0.99         0.99         0.99        1034

In [67]: # SAVE MODEL

pickle.dump( best_model, open('best_spam_model.pkl', 'wb'))

print("Model Saved Successfully")

Model Saved Successfully

In [75]: # CUSTOM EMAIL TEST
sample_email = [ 'Hi Sahil, Your internship meeting is scheduled for tomorrow at 10 AM. Please join using the Zoom link shared earlier' ]

prediction = best_model.predict(sample_email)
if prediction[0] == 1:
    print("Spam Email")
else:
    print("Ham Email")
```

